

ENGR 4420: Software Design and Construction

Armchair GM

Royal Flush

Final Release Report

Team Members

Matt Bennett

Bosit Akhmadjonov

Abby Booker

Liam O'Neill

12/04/25

Table of Contents

Acknowledgment 3

1. Introduction. 3
2. Project Plan. 3
3. Project Requirements. 3
4. Software Design (If this is long, it can be split into 3 sections) 3
 - 4.1 Architectural design. 3
 - 4.2 High level design. 3
 - 4.3 Detailed design. 3
 - 4.4 Design Issues. 3
5. Software Implementation. 4
 - 5.1 Implementation Details. 4
 - 5.2 Software Delivery and Deployment 4
 - 5.3 Implementation and Deployment Issues. 4
6. Conclusion. 4
7. References and Sources. 4
 - Appendix A: Members' Contributions and Learning Points. 5
 - Appendix B: Support documentation. 5
 - Appendix C: Meetings minutes (optional) 5
 - Appendix D: [additional appendices can be included for more information if needed] 5

1. Introduction

Overall description of the project

Armchair GM is a completed full-stack interactive web application that enables baseball fans, fantasy players, and analytics-minded users to construct and visualize MLB rosters. The system allows users to:

- Select any MLB team
- View current rosters
- View detailed player statistics
- Visualize positions using an interactive baseball-diamond UI
- Browse, sort, and sign free agents
- Update rosters dynamically

The final system follows the architecture defined in the SRS: a React frontend, a Node.js/Express backend, communication with the MLB StatsAPI, and a design prepared for future MySQL database integration.

Why this project

Armchair GM was selected because:

- It closely mirrors real-world sports management applications
- It required full-stack development, API integration, and system-level design
- It provided practical experience with asynchronous, data-driven systems
- It aligned well with MLB roster management concepts, making the project both engaging and technically relevant

What you expect to accomplish (main objectives)

The final objectives of Armchair GM were successfully met:

- Deliver an interactive front-end for roster visualization and manipulation
- Implement backend API services for team, player, and free-agent operations
- Integrate live MLB data using the StatsAPI
- Provide a scalable architecture for future simulation, analytics, and database features
- Produce complete software documentation, verification, and validation artifacts

What you expect to learn

By completing this project, the team gained experience in:

- Full-stack software development
- REST API integration and error handling
- Data modeling and UI/UX design
- Managing asynchronous state in React
- Writing verifiable requirements and validating system behavior
- Building modular, maintainable software systems

2. Project Plan

Project scope

The final release of Armchair GM includes:

- React-based roster visualization
- Express backend API
- Integration with MLB StatsAPI
- Free-agent browsing and signing
- Dynamic roster updates
- Player statistics display

Features planned but deferred beyond this release include:

- User accounts
- Persistent roster storage
- Salary simulation
- Trade functionality
- Injury and depth-chart simulation

Main project tasks

- Requirements gathering and SRS creation
- System architecture definition
- UI design and prototyping
- Backend API development
- Frontend roster and stats implementation

- Free-agent system development
- Software verification and testing
- Final documentation and presentation

Project schedule

Week	Task
5–7	Requirements, SRS, planning
8–10	UI wireframes and component structure
9–14	Backend API development
8–14	Frontend roster and diamond UI
11–14	Free-agent panel and signing feature
13–14	System integration and debugging
14	Final documentation and presentation

Risk analysis and mitigation plans

Risk	Impact	Mitigation
MLB StatsAPI downtime	Roster loading failure	Graceful error handling
React state errors	UI inconsistencies	Component isolation
Schedule delays	Reduced features	Core-feature prioritization
Data structure changes	Feature breakage	Validation and normalization
Git merge conflicts	Work stoppage	Clear branching workflow

3. Project Requirements

Overall system functionalities

- Display MLB teams
- Load and visualize rosters
- Display player statistics
- Browse free agents
- Sign free agents
- Update roster positions dynamically

Use cases and scenarios

Major use cases implemented include:

- View Team Roster
- View Player Statistics
- Browse Free Agents
- Sign Free Agent
- Update Position Assignment

(Complete use cases are documented in the SRS.)

Formal requirements specifications

- FR-01: The system shall retrieve MLB rosters using MLB StatsAPI.
- FR-02: The system shall display rosters on a baseball diamond UI.
- FR-03: The system shall allow users to sign free agents to open positions.
- FR-04: The system shall update the roster visually after signing a player.

- FR-05: The system shall fetch player statistics upon user selection.

Development tools/skills requirements

- React
- Node.js / Express
- JavaScript (ES6+)
- CSS layout design
- Axios
- Git / GitHub
- JSON data formats

4. Software Design

4.1 Architectural design

Major components

- React Frontend
- Express Backend API
- MLB StatsAPI
- Free-Agents JSON Data Layer

Integration aspects

The frontend communicates with the backend, which retrieves and normalizes MLB data before returning structured JSON to the UI.

Interaction model

User → React UI → Express Server → MLB StatsAPI → Express Server → React UI

4.2 High level design

How data is stored/used

- Player objects stored in React state
- Free agents loaded from local JSON
- Team identifiers stored as constants
- MySQL planned for future persistence

User interface design

- Team selection dropdown
- Baseball diamond visualization
- Player statistics panel
- Free-agent sidebar
- Position selection modal

Component responsibilities

- TeamSelector: team selection
- DiamondField: position rendering
- PlayerCard: player display
- FreeAgentList: free-agent browsing
- SignModal: signing workflow

4.3 Detailed design

Design objectives

- Modular
- Scalable
- Testable
- API-driven

Modules/components

- Backend: TeamsController, RosterController, StatsController
- Frontend: App.js, TeamSelector, DiamondField, FreeAgentPanel, StatsSidebar

Sequence flow

1. User selects "Sign Player"
2. Modal opens
3. User confirms position
4. POST `/api/roster/update`
5. Backend updates roster
6. UI refreshes

Data models

- Player { id, name, position, stats }
- FreeAgent { id, name, position, war }
- Roster [Player]

4.4 Design Issues

- Incomplete MLB API responses
- React state re-render conflicts
- Manual normalization of free-agent WAR

Deferred features:

- Salary tracking
- Persistent roster saving

5. Software Implementation

5.1 Implementation Details

Development environment

- Windows 10
- Node.js 18+
- VS Code

Tools

- React
- Node.js / Express
- Axios
- GitHub

Connectivity

- Axios GET for StatsAPI
- Axios POST for roster updates

Testing

- UI tests
- API tests via Postman
- Free-agent signing validation

Results

- All implemented features passed testing

Corrective actions

- Improved API error handling
- Normalized free-agent data

5.2 Software Delivery and Deployment

Delivered artifacts

- Express backend
- React frontend build
- free_agents.json
- Documentation

Operational requirements

- Node.js installed
- Internet access

Setup

Npm install

Npm start

5.3 Implementation and Deployment Issues

- Git merge conflicts
- Dependency mismatches
- Deferred salary system

6. Conclusion

Armchair GM successfully delivers a complete roster-management system featuring live MLB data, free-agent signing, and an interactive UI. The final product is reliable, extensible, and well-documented.

Future work

- Database persistence
- Salary simulation
- Trades
- Authentication

7. References and Sources

- [1] MLB StatsAPI Documentation
- [2] React Documentation
- [3] Express.js Documentation
- [4] Axios Documentation
- [5] ENGR 4420 Course Resources